Clean CPU evidence: 0.7.1-clean-4f60380

Measured source `4f60380b4a5ac07931347b4f9eba0ac696983f9a`, tree `f8a34552b825c7f8106a05ec34d7731d6f2538e7`; dirty before/after: false. Library source: workspace 4f60380b4a5ac07931347b4f9eba0ac696983f9a. One pinned CPU; f64 operations and explicitly named f32 constructors; row-major inputs.

Workload / library	Per-call p50 / p95 (us)	Batch-mean p50 / p95 (us)	Rust allocations	Peak additional requested bytes	Process peak RSS (MiB)
cholesky_factor_64_f64/faer	8.145 / 9.587	8.467 / 8.739	2	33280	4.21
cholesky_factor_64_f64/kernix	23.829 / 30.118	24.775 / 25.570	2	65536	4.16
cholesky_factor_64_f64/nalgebra	14.313 / 15.256	14.379 / 14.796	1	32768	3.89
cholesky_factor_64_f64/numpy	10.187 / 10.561	11.133 / 11.960	unavailable	unavailable	33.21
create_ones_64x64_f32/candle	0.236 / 0.250	0.297 / 0.375	7	16696	4.61
create_ones_64x64_f32/faer	0.196 / 0.248	0.186 / 0.206	1	16384	4.06
create_ones_64x64_f32/kernix	0.202 / 0.205	0.208 / 0.221	1	16384	3.99
create_ones_64x64_f32/ndarray	0.154 / 0.156	0.157 / 0.196	1	16384	3.89
create_ones_64x64_f32/numpy	2.441 / 2.672	1.732 / 2.845	unavailable	unavailable	32.88
create_zeros_64x64_f32/candle	0.152 / 0.156	0.158 / 0.168	5	16648	4.42
create_zeros_64x64_f32/faer	0.172 / 0.176	0.168 / 0.172	1	16384	4.14
create_zeros_64x64_f32/kernix	0.138 / 0.140	0.143 / 0.155	1	16384	3.98
create_zeros_64x64_f32/ndarray	0.100 / 0.101	0.098 / 0.104	1	16384	3.95
create_zeros_64x64_f32/numpy	0.386 / 0.432	0.404 / 0.448	unavailable	unavailable	32.88
matmul_128_f64/candle	79.868 / 126.770	89.598 / 102.773	9	131352	5.33
matmul_128_f64/faer	84.244 / 105.219	88.077 / 101.723	1	131072	4.58
matmul_128_f64/kernix	129.254 / 178.132	137.168 / 167.637	3	327680	4.46
matmul_128_f64/nalgebra	126.874 / 172.046	127.316 / 153.569	2	327680	3.98
matmul_128_f64/ndarray	124.491 / 159.399	135.278 / 165.909	2	327680	4.01
matmul_128_f64/numpy	82.029 / 101.136	84.582 / 109.674	unavailable	unavailable	34.56
matmul_32_f64/candle	1.526 / 2.053	1.913 / 2.369	9	8592	4.98
matmul_32_f64/faer	1.783 / 2.487	1.667 / 2.098	1	8192	4.36
matmul_32_f64/kernix	1.533 / 1.825	1.748 / 2.428	3	24576	4.05
matmul_32_f64/nalgebra	2.141 / 2.270	1.602 / 2.329	2	24576	3.92
matmul_32_f64/ndarray	1.497 / 1.572	1.810 / 2.212	2	24576	3.86
matmul_32_f64/numpy	2.829 / 4.447	3.054 / 3.858	unavailable	unavailable	33.15
matmul_batched_4x16x32x16_f64/candle	2.047 / 2.744	2.178 / 2.382	25	25504	5.43
matmul_batched_4x16x32x16_f64/kernix	1.934 / 2.395	2.031 / 2.252	7	16392	4.27
matmul_batched_4x16x32x16_f64/numpy	3.143 / 4.200	3.064 / 3.163	unavailable	unavailable	33.18
matmul_sum_backward_32x16x8_4x4/candle	4.037 / 4.551	4.379 / 4.473	113	27988	5.51
matmul_sum_backward_32x16x8_4x4/kernix	2.413 / 3.061	2.567 / 2.896	16	18376	4.27
qr_256x64_f64/faer	475.510 / 644.549	459.164 / 682.229	7	331776	4.64
qr_256x64_f64/kernix	596.452 / 848.010	627.327 / 808.999	6	557568	4.71
qr_256x64_f64/nalgebra	438.001 / 717.812	463.666 / 512.095	4	295424	4.23
qr_256x64_f64/numpy	399.693 / 498.638	390.477 / 419.301	unavailable	unavailable	34.29
qr_96x32_f64/faer	71.806 / 82.564	76.431 / 97.299	7	67584	4.59
qr_96x32_f64/kernix	48.684 / 54.265	53.467 / 78.751	6	106752	4.24
qr_96x32_f64/nalgebra	42.449 / 62.333	44.089 / 52.466	4	57600	4.01

Workload / library	Per-call p50 / p95 (us)	Batch-mean p50 / p95 (us)	Rust allocations	Peak additional requested bytes	Process peak RSS (MiB)
qr_96x32_f64/numpy	43.914 / 65.815	50.396 / 68.499	unavailable	unavailable	33.46
softmax_256x64_f64/candle	79.667 / 113.744	82.750 / 85.291	37	265200	5.65
softmax_256x64_f64/kernix	85.952 / 91.379	86.228 / 92.049	1	131072	4.44
softmax_256x64_f64/ndarray	72.505 / 86.896	74.884 / 81.223	1	131072	4.29
softmax_256x64_f64/numpy	103.655 / 111.552	104.354 / 111.268	unavailable	unavailable	33.25
training_backward_2x4x8x4_f64/candle	13.327 / 14.807	14.392 / 14.987	472	8332	5.77
training_backward_2x4x8x4_f64/kernix	3.026 / 3.233	3.294 / 3.395	23	4224	4.33
training_forward_2x4x8x4_f64/candle	3.684 / 4.193	3.471 / 3.557	126	3912	5.68
training_forward_2x4x8x4_f64/kernix	2.026 / 2.392	2.250 / 2.286	17	1032	4.33
training_record_2x4x8x4_f64/candle	4.984 / 5.621	5.128 / 5.283	149	10712	5.79
training_record_2x4x8x4_f64/kernix	2.525 / 2.725	2.723 / 2.813	22	6832	4.34
training_sgd_2x4x8x4_f64/kernix	0.536 / 0.566	0.536 / 0.578	5	688	4.53

Method and limits

- Per-call quantiles use nearest ranks over 256 individual warmed calls; batch quantiles use 15 means over separate 10-ms batches. They are separate distributions. Raw samples remain in raw.json.
- Eight warmup calls follow validation. Timing includes allocation, dispatch, computation and output destruction. The first call field excludes output destruction. Inputs and library/runtime imports are prepared outside timing.
- Three extra processes record their first operation calls. These are first-use observations, not cache-flushed or device-startup latency. Timer-pair overhead is reported without subtraction; tiny calls are clock-sensitive.
- ndarray and Candle softmax are explicit stable compositions, not claims about a native softmax API.
- Constructor cases allocate owned 64x64 f32 tensors. Pre-0.6 registry baselines use full(value), explicitly marked in metadata; current workspace runs use zeros/ones. No nested literal reinterpretation is involved.
- All GEMM inputs are C-order and match elementwise. Native output layout differs by library. QR materializes thin Q and R in every implementation. Cholesky returns reusable factors (or NumPy L); no extra factor materialization is timed.
- Kernix checks finite values/symmetry. Some competitors validate a narrower contract. No finite-data precondition check was removed to improve timing. Accuracy results and budgets are retained per worker.
- The backward case uses an actual reverse-mode graph in Kernix and Candle for sum(A @ W), with A constant and W trainable. Both copy input/weight data and returned gradients; no manual derivative is used as a framework ranking.
- Allocations are measured in a separately compiled instrumented binary after warmup, outside all timing loops. Counts cover Rust global-allocator requests, not native allocator internals, stack memory or all NumPy allocations. NumPy allocation counts are unavailable, not zero.
- Requested-byte peak is additional live Rust allocation during one call, including output; it is not scratch-only memory or peak RSS. RSS is the process high-water mark including fixtures, validation and runtime, read from Linux VmHWM. Other platforms may report unavailable.
- The published cases are allocation-inclusive CPU APIs and graph workloads. Pure internal kernel-only and end-to-end application/model latency are not measured. No workload result implies a complete application frame rate.
- faer is explicitly serial; Rayon and BLAS thread counts are one. No GPU comparison. Python/NumPy timings include Python dispatch. Backend and dependency versions/configuration, governor and RAM are recorded.
- Training stages use batched matmul + bias + layer_norm + tanh + step-two feature slice + MSE (Candle index_select). Forward excludes graph recording; record includes parameter/input/target snapshots and eager forward/loss; backward reuses a prepared graph and returns fresh derivatives. Allocation probes measure the corresponding stage, including graph values/scratch, not graph-container-only bytes. SGD uses fixed precomputed real gradients and includes transactional validation/parameter updates; it is not an end-to-end converging training loop. Candle normalization is an explicit population-statistics composition; its

checks/rounding differ. Historical registry baselines without these Tape operations omit the stages, recorded in metadata, rather than substitute manual derivatives.

- A shared laptop has uncontrolled load/frequency/cache effects. Report comparator wins and losses; no universal ranking follows. The source archive comes from committed bytes, with result directories excluded. Library dependencies are pinned by the saved driver lockfile.